Computer Graphics: Lecture 12

Scenes and Interactive Cameras

Slide Deck © Grant Williams, 2026, License: CC-BY-SA 4.0

Welcome back!

Last time we finally learned about perspective.

Our scenes are actually *3D* now.

We learned about the view matrix (what is it?)

We learned about perspective (how is it implemented)

This time

But our camera still isn't interactive.

We're finally going to be implementing a basic WASD camera, like you would find in a level editor.

It's also getting annoying to manage bind groups and scene nodes. We're going to make a simple scene node class.

This lecture follows `sample09` closely.

Be sure to compile it and get it working. You'll need it for project 3!

Scenes

We've learned that Scenes are the main thing that we want to draw.

They represent levels, scenarios, worlds, just collections of 3D objects.

But so far, we've been manually transforming all the matrices, managing the bind groups, and keeping track of vertex buffers.

That changes today. Let's make a proper scene node class.

It turns out, doing this will also make our camera much easier to make.

Scene Nodes

We will learn more about this soon, but Scenes are hierarchical.

They are made up of nodes. A node represents one object with a matrix. However, it doesn't have to be a 3D mesh that gets drawn. There are nodes for many things:

- Cameras
- Lights
- Groups of particles
- Purely for grouping other nodes together (i.e., no data to draw, just a thing that rotates or translates all the child nodes together)
- 3D meshes that get drawn

Scene Nodes (2)

The hierarchical scene node concept is how we can bridge the gap between the needs of the video card and the needs of the game/sim.

For example, a car in a game with destructible cars. It might be made up of many pieces, such as wheels, panels, windows, etc.

The GPU has no concept of the *logical* meaning of a car. It just sees triangles. But we want the pieces of the car to be drawn together, to move together, to be rotated together, etc.

Therefore, we group them all in a hierarchy. Maybe the car has a body node, which has a drivetrain node, which has 4 wheel nodes, which each have a tire node attached to them…

Scene nodes (3)

This hierarchy is what we're building towards.

However, to keep things simple, and to introduce cameras, our scene nodes are going to not be hierarchical until later.

That means each node will represent one thing in the scene (no groups)

[What does something need to exist in the scene?]

A simple scene node

Mainly it needs a matrix.

The matrix describes where the node is. It also describes how big it is, and which way it's facing.

It's a little awkward to mess with the matrix directly, especially when rotation gets involved. Therefore, it's nice to break down the node into 3 components:

1. Position
2. Scale
3. Rotation

A simple scene node (2)

In addition to those three components, a node sometimes represents something that we want to draw.

For example, in a typical video game, the controlled character will have a node. If it's a third person game, there will be one or more meshes attached to that node. If it's a first person game, a hand, weapon, or item will typically be attached to that node.

In our case, we'll have an optional vertex buffer in our node. If it's there, we'll draw it (maybe with a hardcoded number of vertices). If it's not there, we won't draw anything. This isn't ideal, but we'll clean up the software design when we get to meshes.

A simple scene node (3)

So, without further ado, here are our scene node fields:

```
export class SimpleNode {
    yaw: number = 0;
    pitch: number = 0;
    // roll: number = 0; this is for your homework!
    pos: vec3 = vec3.fromValues(0, 0, 0);
    scale: vec3 = vec3.fromValues(1, 1, 1);

    matrix: mat4;
    matrixBuf: GPUBuffer;
    matrixBg: GPUBindGroup;

    mesh: GPUBuffer | undefined;
    name: string; // just for debugging
}
```

First, the mesh and name

Just because we might not have seen this before, I'm going to cover it first: `GPUBuffer | undefined` is something called a **union type**.

It means that that variable, `mesh`, can be an instance of either type. It can either be a `GPUBuffer` or it can be `undefined` (which is both a value and a type, representing the absence of useful information).

This is the most common way to make things "nullable" in Typescript (typically `null` exists, but its type is `Object`, which is not ideal).

Also, `name` is there for debugging. We use it to label the WebGPU data structures so when an error happens we know *which* bind group had the problem.

then, the `pos` and `scale`

We make `pos` and `scale` vector quantities.

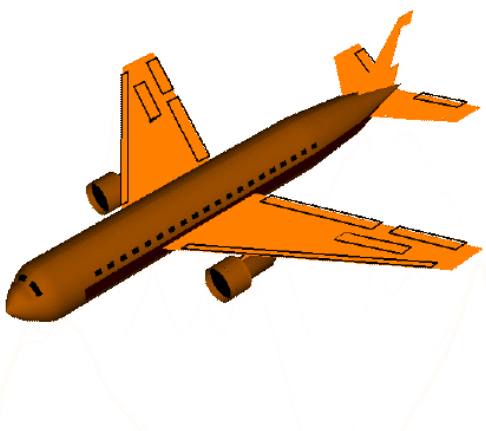
We didn't have to, we could have had `x`, `y`, and `z`, and scalers for each of those, too.

However, it's convenient, because `gl-matrix` has functions that applies translations and scales to a matrix based on a `vec3`.

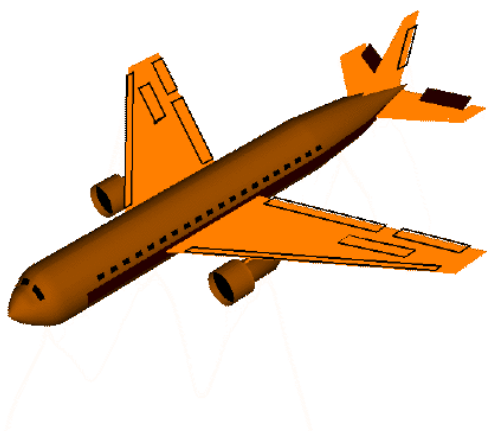
So that just leaves the rotation terms. What are "yaw", "pitch", and "roll"?

Yaw, pitch, roll

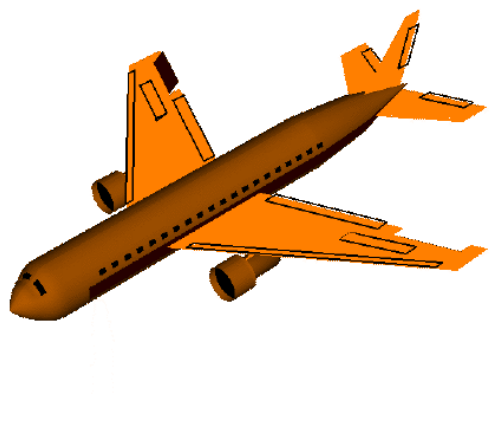
There are three ways an aircraft can rotate:



Yaw



Pitch



Roll

Images by Nancy Hall; Glenn Research Center, NASA, Public Domain

Yaw, pitch, roll (2)

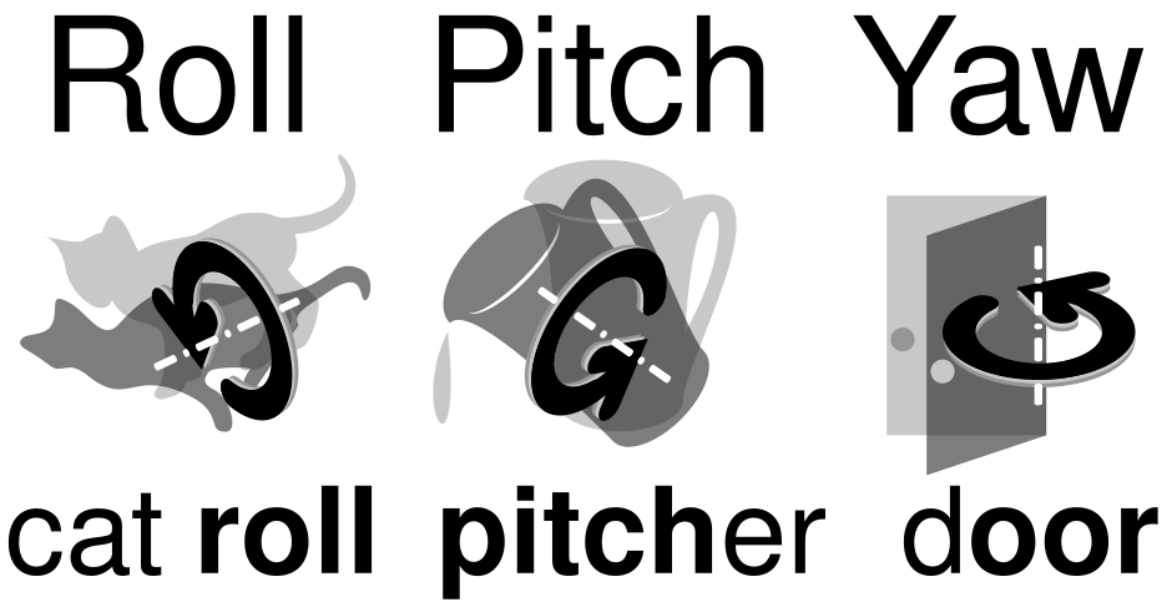
In our terminology:

- Yaw means rotating around the Y axis
- Pitch means rotating around the X axis
- Roll means rotating around the Z axis

Our scene node class is storing the amount of those rotations in turns (it also could store radians or degrees, but I like my rotations to be expressable as a percent).

Of these, I find that roll is easy to remember, but that yaw and pitch are easy to mix up. There's a mnemonic that might be helpful…

Yaw, pitch, roll mnemonic



In my dialect, yaw does not rhyme with door, but that ironically makes it more memorable.

(Image sources: Mnemonic by CMG Lee, Cat images by Chris-martin and Jeremy Ruston, CC-BY-SA 4.0)

Euler and Tait-Bryan angles

Leonhard Euler introduced the practice of using 3-numbers to represent rotations in 3D.

His system was to use the first number to represent how much to rotate around one axis, say Y. Then to use the second number as the amount to rotate around another axis, say, X. Then finally, to perform one more rotation around the new verticle axis created by the first two.

This wasn't ideal for aircraft, whose control surfaces don't rotate around an arbitrary axis but around 3 fixed axes. For this application, the invented by George Bryan is more useful...

Tait-Bryan angles

This is where we apply yaw, pitch, and roll.

Our goal is to obtain a matrix that will do all the camera transformations we want.

Most of them are done for you, but I left out roll, and it turns out, the order matters.

When we push up or down arrows, I want the camera to change pitch.

When we push left or right, I want it to yaw.

For your homework: I want Q to roll left, and E to roll right.

Tait-Bryan angles (2)

Unfortunately, it's not that simple.

Suppose we press E to roll right.

Now I look up. What should happen? There are two logical options:

1. The camera moves like an aircraft. It looks up at an angle.
2. The camera ignores the roll, and looks up around the original x axis instead of the rotated one introduced by the roll.

[I can mime this out]

Tait-Bryan angles (3)

It turns out, the behavior is a consequence of the order we do the rotations. Suppose Y, P, and R are matrices that apply yaw, pitch, roll.

- $P \times Y \times R \times v$ means roll *is independent*. First we roll a vector. Then we yaw it, but the roll already happened, and the yaw isn't affected.. If we roll first, it will change the x and y basis vectors of our camera, so then pitch and yaw will happen around the new roll angle.
- $R \times P \times Y \times v$ means roll around the Z-axis after doing everything else. This will rotate in a confusing direction if you go off the initial axes.

The other permutations

There are actually 6 permutations of orders to apply yaw, pitch and roll.

Technically, any particular orientation is reachable with all of them, but some of them make more or less sense for different applications. Especially when we need to save the state of the previous rotation in order to apply new rotations.

In particular, each one is *different*. Even though you can reach any orientation, the specific inputs you need to enter are different, and some can feel very weird.

Tait-Bryan for flight

Describing aircraft is what this system was invented for, so it works well. The order you probably want to use is: $P \times Y \times R$

That is, first we roll around the Z axis. Then we rotate around the Y axis. Finally we rotate around the X axis.

If we wanted to actually simulate an aircraft, we would need the rotations to happen relative to the actual unique axes of the aircraft, which is axis-angle style rotation, discussed later.

Tait-Bryan for first-person games

For first-person, non-aircraft, applications, let's first consider just pitch and yaw. If we apply pitch after yaw (i.e., P times Y), then *we will rotate around the world's x-axis after twisting, causing us to go off at an angle.*

We don't want it to "cut" left or right along a different angle depending on the angle of our body.

So, our sample uses P times Y:

```
mat4.rotateY(this.matrix, this.matrix, this.yaw * TAU);
mat4.rotateX(this.matrix, this.matrix, this.pitch * TAU);
```

Remember, these are world axes

`rotateY` means “rotate around the Y axis”.

If you do that *before* rotating around X, then, you will end up rotating around X at an angle.

Just to be clear, I have named the matrices Y, P, and R because they correspond to those operations when used *in this order*. If you move the Y matrix out of order, it’s no longer a Yaw matrix. It’s a “rotate wrong” matrix.

If you want a matrix that will apply yaw to the object’s local coordinate system and not a global rotation, you need an axis-angle rotation, described later in this lecture.

What about roll, translation, and scaling?

For your project, I want it to feel like a flying camera in an FPS game.

That is, roll needs to happen first.

What about translation? It has to happen last. If we translate first, we will apply our rotations around the origin, which will cause us to orbit the center of the world.

And scaling? It should happen first, or at least, before translation. Otherwise we'll end up scaling relative to the origin instead of locally.

So the final order for your project is TYPRS. Just remember "typers" as a mnemonic. And then remember that those are matrices!

Cameras are just nodes

Note that these terms apply to every node, not only the camera.

We can change the yaw of the camera, but we can also change the yaw of a car in the scene.

The difference is that, when we're done, the camera will have its matrix inverted to create the view matrix. And that matrix will be combined with the projection matrix.

Updating nodes

For any node, once we compute the roll, pitch, yaw, position, and scale, we update the matrix stored in the node's bind group. This makes sure that when we process the node, the matrix in the shader is correct.

```
updateMatrix(device: GPUDevice): void {
    mat4.identity(this.matrix);
    mat4.translate(this.matrix, this.matrix, this.pos);
    mat4.rotateY(this.matrix, this.matrix, this.yaw * TAU);
    mat4.rotateX(this.matrix, this.matrix, this.pitch * TAU);
    mat4.scale(this.matrix, this.matrix, this.scale);
    device.queue.writeBuffer( // overwrite the matrix buffer
        this.matrixBuf, 0,
        new Float32Array(this.matrix));
    }
}
```

Initializing nodes

The only things we have to do when initializing a node is to are:

- Create a matrix locally
- Create a buffer that will store the matrix on the GPU
- Create a bind-group that points to that buffer

Check the sample to see exactly how this looks. It's a little too big to fit in a slide.

This is a big improvement. Previously, we stored our bind groups and buffers separately. Now, they are combined in one place. Every node needs a bind group that points to its own matrix data, so it's much more convenient to make a more complicated scene, now.

Questions?

Camera vectors

The camera's model matrix tells us a lot about it:

$$\begin{pmatrix} R_x & U_x & B_x & T_x \\ R_y & U_y & B_y & T_y \\ R_z & U_z & B_z & T_z \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Those basis vectors are very important. Each one tells us something about the camera's orientation or position.

For instance, the B vector stands for "backward". It literally points backward, in the direction opposite the camera is facing.[1]

[1] If we used left-handed coordinates it would be the "forward" vector instead. IMO left handed coordinates are nicer.

Camera vectors (2)

The U vector stands for “up”. It points in the upward direction relative to the pitch of the camera.

The R vector points right.

The T vector is the position (translation) of the camera.

This isn’t just theoretical. If we want to move backward, we don’t want to just add (0, 0, 1) to the camera’s position. *That will make it so that it always moves relative to the z-axis instead of taking into account the direction we’re facing.*

Instead, we want to grab the backward vector and move that way.

Camera storage

To do this, the most straightforward way is to literally just pull the values out of the matrix (the gl-matrix data, not the GPU buffer, those are harder to access).

Let's refer to a matrix's elements the way mathematicians do[1]:

$$\begin{pmatrix} m_{00} & m_{01} & m_{02} & m_{03} \\ m_{10} & m_{11} & m_{12} & m_{13} \\ m_{20} & m_{21} & m_{22} & m_{23} \\ m_{30} & m_{31} & m_{32} & m_{33} \end{pmatrix}$$

That is, the first subscript number is the row, the second is the column.

[1] technically mathematicians count from 1, but we can't totally forsake our customs.

Camera storage (2)

Within gl-matrix, the 4-by-4 matrices are secretly an array of 16 values.

You might think they would be *these* values:
`[m00, m01, m02, m03, m10, m11, m12, ...]`

This would be **row major** ordering. This is the typical ordering in most programming languages. We store a 2D array as a list of rows…

This is not how WebGPU (or OpenGL) work. They are **column major** for matrix storage. This means the first *column* comes first:
`[m00, m10, m20, m30, m01, m11, m21, ...]`

Camera storage (3)

Therefore, if we want to pull the backward vector out of the camera's matrix, we need to do it lke this:

```
backward(): vec3 {
    return vec3.fromValues(
      this.matrix[8], this.matrix[9], this.matrix[10]);
}
```

Notice, we got values 8, 9, and 10. Not values 2, 6, 10 like it would have been if it were row-major.

But what do we do with the backward vector?

Moving forward

To move forward, we need these pieces of information:

- The backward/forward vector
- The camera's position
- The camera's speed (a constant that you pick)
- The timestep (the time since the last update)

We just discussed the backward vector.

The camera's position is the last column (the w-column) of the camera's matrix. Alternatively, you can store it separately like I am.

The speed is just a number with units "units per second".

The timestep

The timestep is how much time has passed since the last frame.

This is surprisingly complex, and it's easy to make mistakes.

If you are simulating physics, you want a *fixed timestep*. To learn why, this is a classic, heavily referenced article in game development: Fix Your Timestep.

Fixing a timestep means processing updates, such as positions or velocity changes, every so often (like 100 times per second or so), on a fixed cycle.

The timestep (2)

However, for purely graphical effects, we typically *don't* do this.

Instead, we calculate how much time has passed since the last frame.

To do this, we need to store when the last frame was *started* (**not** finished! this is one of the mistakes that is very common).

We compute the time delta between the start of the current frame and the start of the last frame. In JavaScript, the unit is milliseconds, but I prefer to use seconds, so I divide by 1000.

This time delta, in seconds, is used to determine how much to move.

Moving forward (2)

What we need to do, is compute the total movement amount.

Then scale the backward vector by the opposite amount.

Finally, add that scaled vector to the cameras position.

Something like this:

$$p' = B \times -\text{speed} \times \delta t + p$$

That is, the new position is the old one, plus the backward vector, times the speed, times the change in time (in seconds)

[How do you think we move to the side?]

Other camera movements

What about rotation?

Those are a bit easier, we just increase or decrease the corresponding yaw, pitch, or roll.

But we do so based on how much time has passed. We have a rotation speed, which can be different than the movement speed. Its units can be “turns per second” or “degrees per second” or “radians per second”.

Rotation is always around the camera, so there’s no translation we have to add.

Questions?

Detecting keys

There is one last, but critically important, thing we need to discuss.

How do we actually know when the user is pressing a button?

JavaScript has an API for this, but it's slightly more involved than just reading the keyboard.

In general, browsers are extremely concerned with privacy. They certainly are not going to let you read which keys are down in the person's computer. What if your app is running while they tab over and enter a password into their bank?

Detecting keys (2)

Instead, JavaScript allows you to detect events.

Events cover an enormous amount of user behavior, but among them are “keydown” and “keyup”.

You will only receive events when your tab is active. The user has to be hitting keys while using your browser app, so you can’t spy on them.

Therefore, we have to internally track which buttons are down or up so that we can, e.g., move forward as long as the forward button is down.

We certainly don’t want to move forward a fixed amount every time the user taps W!

Detecting keys (3)

I created this class for storing keys. It will continue to be useful.

```
export class Keys {
    down: Set<string> = new Set();
    constructor() {
        addEventListener('keydown', (event) => {
            this.down.add(event.code);
        });
        addEventListener('keyup', (event) => {
            this.down.delete(event.code);
        });
    }
    isDown(code: string): boolean {
        return this.down.has(code);
    }
}
```

Detecting keys (4)

`down` is a `Set<string>`. That is, it's a hashset that stores strings.

The strings refer to *key codes*. We'll discuss those shortly.

In the constructor, we use the `addEventListener` function, which is built-in to web-facing JavaScript.

This function takes a kind of event, and a function. It will call the function you give it and pass the event, whenever that kind of event occurs.

We use anonymous function notation. `(event) => { ... }` refers to a function that takes a parameter named `event`.

Detecting keys (5)

When we receive a `keydown` event, our function inserts the key's name into the hashset.

When we receive a `keyup` event, our function removes that key.

Using this system, we can quickly tell whether a key is down by whether or not it is in the hashset.

Note: if the user is on a different tab or not using their browser, you will not receive an event. Therefore, if they press "W" and then tab out, the camera will keep moving forward.[1]

[1] It's possible to detect this situation (it's called 'Focus Loss'), but these examples are meant to be very simple.

Key codes

A key code is kind of like a unique name for a key, but *it doesn't change* even if the keyboard layout changes.

That is, on a standard US QWERTY keyboard, W is the second letter from the top left.

But in a French AZERTY layout, there's a Z instead. In fact, the W on the French layout is where the Z is on the American layout.

Keycodes are based on QWERTY. So by using the keycode, Americans can use WASD, and French speakers can use ZQSD, and it will work without rebinding.

Keyboard layouts

You might be worried about this. How many layouts are there?

Most of the world uses QWERTY, even languages with different writing systems will typically have Roman letters in QWERTY layout. Here's the Japanese layout as an example.

However, French uses AZERTY. German uses QWERTZ. And Cyrillic languages typically use some modification of JCUKEN.

Regardless, use keycodes to specify the default. Then the others are easy to derive. The W character on a QWERTY keyboard will have the same code as a Z in AZERTY or a C in JCUKEN.

Detecting Keys

Every time we update our frame, we just check every key.

You could store your key actions in a dictionary if you wanted. This can be cleaner than a bunch of if-statements and also enable customization.

But, if-statements are a simple starting point:

```
if (this.keys.isDown('KeyW')) {
    vec3.scaleAndAdd(this.camera.pos, this.camera.pos,
      this.camera.backward(), -CAM_MOVE_SPEED * dt);
} // we negate to make backward be forward
```

(we can use `scaleAndAdd` to efficiently scale the vector and add it)

Questions?

Other systems of rotation

Using fixed-axis rotations like this is not the only way to store a camera.

It's simple, and makes it easy to debug and control the camera.

But, depending on the application, it's not even the *best* system.

The ultimate goal is to end up with a matrix, but there are a couple of ways we can do that…

Axis-angle rotations

We haven't discussed it, but there's actually a fairly complex rotation matrix which can rotate around an arbitrary axis.

This can be nice for, e.g., a space sim, where the fixed 3-axis rotations are inappropriate.

Why are they a problem? Because of something called **Gimbal Lock**

Gimbal lock

Pretend that you are using the 3-fixed-axes we talked about earlier. You have a pitch, a yaw, and a roll.

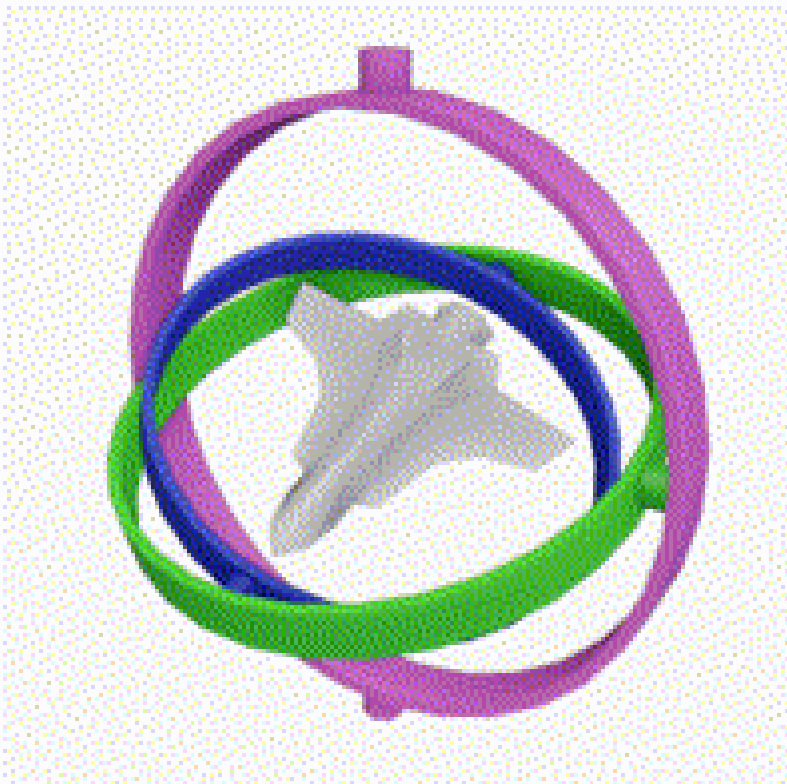
Try this (once you read it all, since you'll be facing the ceiling):

- Pitch up until you're looking up
- Now, rotate right and left. Notice that you spin around.
- Now, *roll* right and left. Notice that it's exactly the same as rotating

At this point, you're sort of stuck. You can't rotate towards your shoulders anymore. You've lost a degree of freedom. The only way to fix it is to "unpitch" yourself first, then rotate.

This condition is called **gimbal lock**.

Gimbal lock (2)



A gimbal getting locked. Animation by DrummyFish, public domain.

Gimbal lock (3)

In some situations we don't have to worry about gimbal lock.

Most first person games never let you rotate all the way up to look perfectly vertically. Therefore, they avoid the problem.

This is actually a pretty big restriction on range of motion: most people can rotate so far that we can look backwards, but most games won't let you do that.

But what if your app requires true free rotation? We're in space, or some kind of 3D maze environment where we actually need 3 degrees of freedom always? Then we don't use 3 fixed scalars...

Axis-angle rotations(2)

One option is to rotate around an arbitrary axis.

Which axis?

Usually, the camera's axes. Instead of rotating around the global X, Y, and Z axes, we can use the camera vectors.

This avoids the problem.



Hellbender

Quaternion Cameras

Alternatively, we could use quaternions instead.

A quaternion can encode a direction *and* a magnitude.

Quaternions conveniently encode arbitrary rotations.

You would still use a vector for the position.

The different columns of the matrix can be reconstructed using the Euler-Rodriguez formula.

Afterwards, we can set the w-column to be the camera's position.

But there's another way...

Quaternion Cameras (2)

Applying a quaternion is a little weird. To rotate a vector by a quaternion, you transform a vector ‘v’ with: qvq^{-1}

Where that last factor is the “conjugate” of the quaternion (i.e., the quaternion with the sign flipped for the imaginary parts)

The interesting thing is that you can apply this transformation to the standard axes (e.g., (1, 0, 0), (0, 1, 0), and (0, 0, 1)) to get the right, up, and backward/forward vectors of the camera.

In practice…it’s easier to use a matrix. The shader doesn’t have built-in quaternion operators, for example.

Today's summary

It was a whirlwind today, but now you're ready for the project.

We learned how to make interactive cameras, how to encode rotations, avoid gimbal lock, and move around.

We also learned how to make a simple scene, with nodes that store the matrices of each thing we want in our scene.

Be sure to build the sample and read the book (mindful of the fact that the NDC z-range is actually $[0, 1]$, not $[-1, 1]$).

Questions?